Introduction to C++

- C++
  - Bjarne Stroustrup (Bell Labs, 1979)
  - started as extension to C (macros and variables)
  - added new useful, features
  - nowadays a language of its own
  - C++ (the next thing after C, though wouldn’t ++C be more appropriate?)

Outline

- Intro to C++
  - Object-Oriented Programming
  - Changes in C++
    - comments
    - variable declaration location
    - initialization
    - pointer changes
    - tagged structure type
    - enum types
    - bool type

Object-Oriented Programming

- First-class objects - atomic types in C
  - int, float, char
  - have:
    - values
    - sets of operations that can be applied to them
  - how represented irrelevant to how they are manipulated
- Other objects - structures in C
  - cannot be printed
  - do not have operations associated with them (at least, not directly)

Object-Oriented Idea

- Make all objects, whether C-defined or user-defined, first-class objects
- For C++ structures (called classes) allow:
  - functions to be associated with the class
  - only allow certain functions to access the internals of the class
  - allow the user to re-define existing functions (for example, input and output) to work on class

Classes of Objects in C++

- Classes
  - similar to structures in C (in fact, you can can still use the struct definition)
  - have fields corresponding to fields of a structure in C (similar to variables)
  - have fields corresponding to functions in C (functions that can be applied to that structure)
  - some fields are accessible by everyone, some not (data hiding)
  - some fields shared by the entire class

Instances of Classes in C++

- A class in C++ is like a type in C
- Variables created of a particular class are instances of that class
- Variables have values for fields of the class
- Class example: Student
  - has name, id, gpa, etc. fields that store values
  - has functions, changeGPA, addCredits, that can be applied to instances of that class
- Instance examples: John Doe, Jane Doe
  - each with their own values for the fields of the class

Comments in C++

- Can use C form of comments /* A Comment */
- Can also use // form:
  - when // encountered, remainder of line ignored
  - works only on that line
- Examples:

```
void main() {
   int I;  // Variable used in loops
   char C; // No comment comment
```

Variable Declarations

- In C++, variable declarations are not restricted to the beginnings of blocks (before any code)
  - you may interleave declarations/statements as needed
  - it is still good style to have declarations first
- Example

```
void main() {
  int I = 5;
  printf("Please enter J: ");
  int J; // Not declared at the start
  scanf("%d",&J);
```

Counter Variables in a For Loop

- You can declare the variable(s) used in a for loop in the initialization section of the for loop
  - good when counter used in for loop only exists in for loop (variable is throw-away)
- Example

```
for (int I = 0; I < 5; I++)
   printf("%d\n",I);
```

- Variable exists only during for loop (goes away when loop ends)

Initializing Global Variables

- Not restricted to using constant literal values in initializing global variables, can use any evaluable expression
- Example:

```
int rows = 5;
int cols = 6;
int size = rows * cols;

void main() {
...
```

Initializing Array Elements

- When giving a list of initial array values in C++, you can use expressions that have to be evaluated
- Values calculated at run-time before initialization done
- Example:

```
void main() {
  int n1, n2, n3;
  int *nptr[] = { &n1, &n2, &n3 };
```

void*

- In C it is legal to cast other pointers to and from a void *
- In C++ this is an error, to cast you should use an explicit casting command
- Example:

```
int N;
int *P = &N;
void *Q = P;            // illegal in C++
void *R = (void *) P; // ok
```

NULL in C++

- C++ does not use the value NULL, instead NULL is always 0 in C++, so we simply use 0
- Example:

```
int *P = 0; // equivalent to
            // setting P to NULL
```

- Can check for a 0 pointer as if true/false:

```
if (!P)  // P is 0 (NULL)
   ...
else // P is not 0 (non-NULL)
   ...
```

Tags and struct

- When using struct command in C++ (and for other tagged types), can create type using tag format and not use tag in variable declaration:

```
struct MyType {
  int A;
  float B;
};
MyType V;
```

enum in C++

- Enumerated types not directly represented as integers in C++
  - certain operations that are legal in C do not work in C++
- Example:

```
void main() {
   enum Color { red, blue, green };
   Color c = red;
   c = blue;
   c = 1;  // Error in C++
   ++c;  // Error in C++
```

bool

- C has no explicit type for true/false values
- C++ introduces type bool (later versions of C++)
  - also adds two new bool literal constants true (1) and false (0)
- Other integral types (int, char, etc.) are implicitly converted to bool when appropriate
  - non-zero values are converted to true
  - zero values are converted to false

bool operations

- Operators requiring bool value(s) and producing a bool value:

  && (And), || (Or), ! (Not)
- Relational operators (==, !=, <, >, <=, >=) produce bool values
- Some statements expect expressions that produce bool values:

```
if (boolean_expression)
 while (boolean_expression)
 do … while (boolean_expression)
 for ( ; boolean_expression; )
```

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called ***functions***.	In object-oriented programming, the program is divided into small parts called ***objects***.
Procedural programming follows a ***top-down approach***.	Object-oriented programming follows a ***bottom-up approach***.
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected, etc.
Adding new data and functions is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way of hiding data so it is ***less secure***.	Object-oriented programming provides data hiding so it is ***more secure***.
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.
In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
In procedural programming, the function is more important than the data.	In object-oriented programming, data is more important than function.
Procedural programming is based on the ***unreal world***.	Object-oriented programming is based on the ***real world***.
Procedural programming is used for designing medium-sized programs.	Object-oriented programming is used for designing large and complex programs.
Procedural programming uses the concept of procedure abstraction.	Object-oriented programming uses the concept of data abstraction.
Code reusability absent in procedural programming,	Code reusability present in object-oriented programming.
Examples: C, FORTRAN, Pascal, Basic, etc.	**Examples:** C++, Java, Python, C#, etc.

The New C++ Headers

As you know, when you use a library function in a program, you must include its header. This is done using the **#include** statement. For example, in C, to include the header for the I/O functions, you include **stdio.h** with a statement like this:

```
#include <stdio.h>
```

Here, **stdio.h** is the name of the file used by the I/O functions, and the preceding statement causes that file to be included in your program. The key point is that this **#include** statement normally *includes a file*.

When C++ was first invented and for several years after that, it used the same style of headers as did C. That is, it used *header files*. In fact, Standard C++ still supports C-style headers for header files that you create and for backward compatibility. However, Standard C++ created a new kind of header that is used by the Standard C++ library. The new-style headers *do not* specify filenames. Instead, they simply specify standard identifiers that may be mapped to files by the compiler, although they need not be. The new-style C++ headers are an abstraction that simply guarantee that the appropriate prototypes and definitions required by the C++ library have been declared.

Since the new-style headers are not filenames, they do not have a **.h** extension. They consist solely of the header name contained between angle brackets. For example, here are some of the new-style headers supported by Standard C++.

<iostream> <fstream> <vector> <string>

The new-style headers are included using the **#include** statement. The only difference is that the new-style headers do not necessarily represent filenames.

Because C++ includes the entire C function library, it still supports the standard C-style header files associated with that library. That is, header files such as **stdio.h** or **ctype.h** are still available. However, Standard C++ also defines new-style headers that you can use in place of these header files. The C++ versions of the C standard headers simply add a "c" prefix to the filename and drop the **.h**. For example, the C++ new-style header for **math.h** is **<cmath>**. The one for **string.h** is **<cstring>**. Although it is currently permissible to include a C-style header file when using C library functions, this approach is deprecated by Standard C++ (that is, it is not recommended). For this reason, from this point forward, this book will use new-style C++ headers in all **#include** statements. If your compiler does not support new-style headers for the C function library, then simply substitute the old-style, C-like headers.

Since the new-style header is a relatively recent addition to C++, you will still find many, many older programs that don't use it. These programs employ C-style headers, in which a filename is specified. As the old-style skeletal program shows, the traditional way to include the I/O header is as shown here.

```
#include <iostream.h>
```

This causes the file **iostream.h** to be included in your program. In general, an old-style header file will use the same name as its corresponding new-style header with a **.h** appended.

As of this writing, all C++ compilers support the old-style headers. However, the old-style headers have been declared obsolete and their use in new programs is not recommended. This is why they are not used in this book.

Remember *While still common in existing C++ code, old-style headers are obsolete.*

Namespaces

When you include a new-style header in your program, the contents of that header are contained in the **std** namespace. A *namespace* is simply a declarative region. The purpose of a namespace is to localize the names of identifiers to avoid name collisions. Elements declared in one namespace are separate from elements declared in another. Originally, the names of the C++ library functions, etc., were simply put into the global namespace (as they are in C). However, with the advent of the new-style headers, the contents of these headers were placed in the **std** namespace. We will look closely at namespaces later in this book. For now, you won't need to worry about them because the statement

```
using namespace std;
```


brings the **std** namespace into visibility (i.e., it puts **std** into the global namespace). After this statement has been compiled, there is no difference between working with an old-style header and a new-style one.

One other point: for the sake of compatibility, when a C++ program includes a C header, such as **stdio.h**, its contents are put into the global namespace. This allows a C++ compiler to compile C-subset programs.